

PRACTICAL NO 8

Aim :- K-Nearest Neighbors (K-NN)

1. Implement K-NN algorithm for classification or regression.
2. Apply K-NN to the Red Wine Quality dataset and predict class values for test data.
3. Evaluate accuracy/error and analyze the results.

Step	Method / Function Used	Purpose	
1	<code>pd.read_csv()</code>	Loads the Red Wine Quality dataset from the CSV file.	
2	<code>drop()</code>	Separates the input features from the target column.	
3	<code>train_test_split()</code>	Splits the dataset into training and testing data in an 80:20 ratio.	
4	<code>StandardScaler()</code>	Standardizes the feature values before applying K-NN.	
5	<code>KNeighborsClassifier()</code>	Creates the K-NN classification model using K = 5.	
6	<code>fit()</code>	Trains the K-NN model using the training data.	
7	<code>predict()</code>	Predicts wine-quality classes for the test data.	
8	<code>accuracy_score()</code>	Calculates the classification accuracy.	
9	<code>1 - accuracy</code>	Calculates the prediction error.	
10	<code>classification_report()</code>	Displays precision, recall and F1-score.	
11	<code>confusion_matrix()</code>	Creates the confusion matrix for actual and predicted classes.	
12	<code>plt.imshow()</code>	Displays the confusion matrix graphically.	
13	<code>KNeighborsClassifier()</code> in loop	Tests different K values from 1 to 15.	

14	<code>accuracy_score()</code> in loop	Calculates accuracy for each K value.	
15	<code>plt.plot()</code>	Plots K value against accuracy.	
16	<code>max()</code> and <code>index()</code>	Finds the K value having the highest accuracy.	

CODE:

```

T090_PRACTICAL8_Alpy - C:\practical\AI\T090_PRACTICAL8_Alpy (3.13.0)
File Edit Format Run Options Window Help
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

data = pd.read_csv("winequality-red.csv")

print("First 5 Rows:")
print(data.head())

print("\nDataset Shape:")
print(data.shape)

print("\nDataset Columns:")
print(data.columns)

X = data.drop("quality", axis=1)
y = data["quality"]

print("\nNumber of Features:")

```

Activate Windows
Go to Settings to activate Windows.

29°C
Partly cloudy

Search

Ln: 235 Col: 0

19:27
03-09-2026

```
T090_PRACTICAL8_Alpy - C:\practical\AI\T090_PRACTICAL8_Alpy (3.13.0)
File Edit Format Run Options Window Help
print("\nNumber of Samples:")
print(X.shape[0])

print("\nTarget Classes:")
print(sorted(y.unique()))

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("\nTraining Samples:")
print(X_train.shape[0])

print("\nTesting Samples:")
print(X_test.shape[0])

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\nData Standardization Completed!")

T090_PRACTICAL8_Alpy - C:\practical\AI\T090_PRACTICAL8_Alpy (3.13.0)
File Edit Format Run Options Window Help
knn = KNeighborsClassifier(n_neighbors=5)

knn.fit(X_train_scaled, y_train)

print("\nK-NN Model Trained Successfully!")

y_pred = knn.predict(X_test_scaled)

print("\nFirst 15 Actual Values:")
print(y_test.values[:15])

print("\nFirst 15 Predicted Values:")
print(y_pred[:15])

accuracy = accuracy_score(y_test, y_pred)
error = 1 - accuracy

print("\nAccuracy:")
print(accuracy)

print("\nAccuracy Percentage:")
print(accuracy * 100, "%")
```

```

T090_PRACTICAL8_Alpy - C:\practical\AI\T090_PRACTICAL8_Alpy (3.13.0)
File Edit Format Run Options Window Help

print("\nPrediction Error:")
print(error)

print("\nPrediction Error Percentage:")
print(error * 100, "%")

print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    zero_division=0
))

cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)

classes = sorted(y.unique())

plt.figure(figsize=(8, 6))
plt.imshow(cm, interpolation="nearest")
plt.title("K-NN Confusion Matrix")

T090_PRACTICAL8_Alpy - C:\practical\AI\T090_PRACTICAL8_Alpy (3.13.0)
File Edit Format Run Options Window Help

plt.xlabel("Predicted Quality")
plt.ylabel("Actual Quality")
plt.xticks(
    range(len(classes)),
    classes
)
plt.yticks(
    range(len(classes)),
    classes
)
for i in range(len(cm)):
    for j in range(len(cm)):
        plt.text(
            j,
            i,
            str(cm[i, j]),
            ha="center",
            va="center"
        )
plt.colorbar()
plt.tight_layout()
plt.show()

```

```
*T090_PRACTICAL8_Alpy - C:\practical\AI\T090_PRACTICAL8_Alpy (3.13.0)*
File Edit Format Run Options Window Help
k_values = range(1, 16)

accuracy_values = []

for k in k_values:

    knn_model = KNeighborsClassifier(
        n_neighbors=k
    )

    knn_model.fit(
        X_train_scaled,
        y_train
    )

    y_k_pred = knn_model.predict(
        X_test_scaled
    )

    k_accuracy = accuracy_score(
        y_test,
        y_k_pred
    )

    accuracy_values.append(
        k_accuracy
    )

best_index = accuracy_values.index(
    max(accuracy_values)
)

print("\nBest K Value:")
print(best_k)

print("\nBest K Accuracy:")
print(best_accuracy)

print("\nBest K Accuracy Percentage:")
print(best_accuracy * 100, "%")

plt.figure(figsize=(8, 5))

plt.plot(
    k_values,
    accuracy_values,
    marker="o"
)

plt.xlabel("K Value")
plt.ylabel("Accuracy")
plt.title("K-NN: K Value vs Accuracy")
plt.xticks(k_values)
plt.grid(True)
plt.show()
```

Activate Windows
Go to Settings to activate Windows.

29°C
Partly cloudy

Ln: 162 Col: 10

19:31
03-09-2026

Activate Windows
Go to Settings to activate Windows.

29°C
Partly cloudy

Ln: 162 Col: 10

19:32
03-09-2026

SHETH L.U.J. AND SIR M.V. COLLEGE

OUTPUT:

```
Python 3.13.0 (tags/v3.13.0:60403a5, Oct 7 2024, 09:38:07) [MSC v.1941 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/practical/AI/T090_PRACTICAL8_AI.py =====
First 5 Rows:
  fixed acidity  volatile acidity  citric acid  ...  sulphates  alcohol  quality
0             7.4             0.70       0.00  ...      0.56       9.4         5
1             7.8             0.88       0.00  ...      0.68       9.8         5
2             7.8             0.76       0.04  ...      0.65       9.8         5
3            11.2             0.28       0.56  ...      0.58       9.8         6
4             7.4             0.70       0.00  ...      0.56       9.4         5

[5 rows x 12 columns]

Dataset Shape:
(1599, 12)

Dataset Columns:
Index(['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar',
      'chlorides', 'free sulfur dioxide', 'total sulfur dioxide', 'density',
      'pH', 'sulphates', 'alcohol', 'quality'],
      dtype='str')

Number of Features:
11

Number of Samples:
1599

Target Classes:
[np.int64(3), np.int64(4), np.int64(5), np.int64(6), np.int64(7), np.int64(8)]

Number of Samples:
1599

Target Classes:
[np.int64(3), np.int64(4), np.int64(5), np.int64(6), np.int64(7), np.int64(8)]

Training Samples:
1279

Testing Samples:
320

Data Standardization Completed!

K-NN Model Trained Successfully!

First 15 Actual Values:
[6 5 6 5 6 5 5 5 5 6 7 3 5 5 6]

First 15 Predicted Values:
[5 5 6 6 6 5 5 5 6 6 7 6 6 5 5]

Accuracy:
0.546875

Accuracy Percentage:
54.6875 %

Prediction Error:
0.453125
```

SHETH L.U.J. AND SIR M.V. COLLEGE

```

IDLE Shell 3.13.0*
File Edit Shell Debug Options Window Help

Accuracy Percentage:
54.6875 %

Prediction Error:
0.453125

Prediction Error Percentage:
45.3125 %

Classification Report:
              precision    recall  f1-score   support

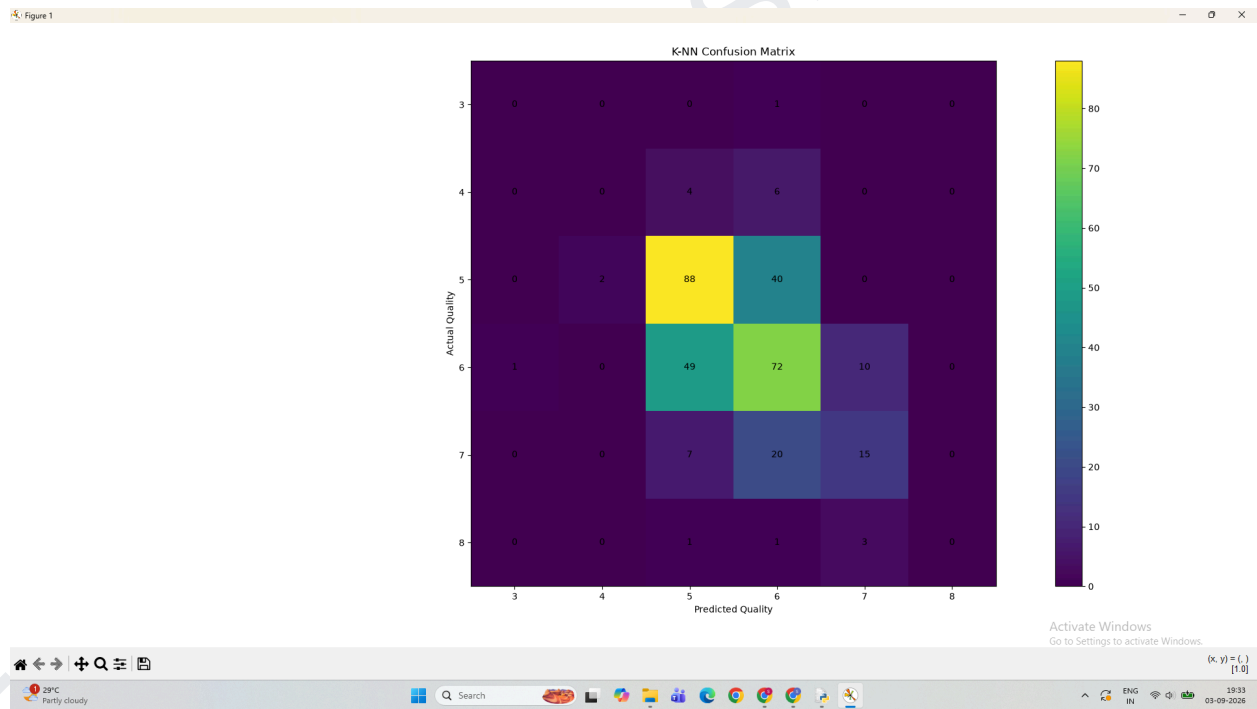
         3            0.00        0.00        0.00         1
         4            0.00        0.00        0.00        10
         5            0.59        0.68        0.63       130
         6            0.51        0.55        0.53       132
         7            0.54        0.36        0.43        42
         8            0.00        0.00        0.00         5

   accuracy            0.55            320
  macro avg            0.27            320
 weighted avg            0.52            320

Confusion Matrix:
[[ 0  0  0  1  0  0]
 [ 0  0  4  6  0  0]
 [ 0  2 88 40  0  0]
 [ 1  0 49 72 10  0]
 [ 0  0  7 20 15  0]
 [ 0  0  1  1  3  0]]

```

GRAPH:



SHETH L.U.J. AND SIR M.V. COLLEGE

